# 추상 클래스와 인터페이스

금오공과대학교
컴퓨터공학과

# 학습 목표

- 추상 메소드와 추상 클래스의 필요성과 용도를 설명할 수 있다.
- 추상 클래스의 선언 방법과 특성을 설명할 수 있다.
- 인터페이스의 용도와 사용 방법을 설명할 수 있다.

# 추상 메소드abstract method

- 선언되어 있으나 구현되어 있지 않은 메소드
- 추상 메소드 선언
  - abstract 키워드 사용
  - ex) `public abstract int getValue();`
- 서브 클래스에서 재정의overriding하여 구현해야 함

# 추상 클래스abstract class

1. 추상 메소드를 하나라도 가진 클래스
   - 클래스 앞에 반드시 abstract라고 선언해야 함

2. 추상 메소드가 하나도 없지만 클래스 앞에 abstract로 선언한 경우

# 추상 클래스의 두 가지 형태

```
// 1. 추상 메소드를 가진 추상 클래스

abstract class Shape {
  public String name;

  public Shape() { name = null; }
  abstract public void draw(); // 추상 메소드 선언
}
```

```
// 2. 추상 메소드 없는 추상 클래스

abstract class Person {
  public String name;

  public Person(String name) { this.name = name; }
  public String getName() { return name; }
}
```

# 추상 클래스 필요성

- 계층적 상속 관계를 갖는 클래스 구조를 만들 때
- 설계와 구현 분리
  - 슈퍼 클래스에서는 개념적 특징 정의 (규격 마련)
  - 서브 클래스에서 구체적 행위 구현
- 설계의 동일성 확보
  - 클래스 설계자가 여러 명인 경우 필드명과 메소드명을 동일하게 지정 가능

# 추상 클래스 특성

- 추상 클래스의 객체는 생성 불가
  - 레퍼런스 변수 타입으로 사용 가능

- 추상 클래스의 상속
  - 추상 클래스를 상속할 때 추상 메소드 미구현
    ⇒ 서브 클래스도 추상 클래스 됨
    - 서브 클래스도 abstract로 선언해야 함

# 추상 클래스의 인스턴스 생성 불가

```
abstract class Shape { // 추상 클래스 선언
  public String name;

  public Shape() { name = null; }
  abstract public void draw(); // 추상 메소드 선언
}

public class ShapeTest {
  public static void main(String [] args) {
    Shape s;
    s = new Shape(); // 컴파일 오류
  }
}
```

```
Exception in thread "main" java.lang.Error: Unresolved compilation problem:
        Cannot instantiate the type DObject
```

# 추상 클래스의 활용 예

```
class Shape {
  protected String name;

  public Shape() { next = null;}
  public void draw() {
    System.out.print("shape");
  }
}
```

추상 클래스

```
abstract class Shape {
  protected String name;

  public Shape() { name = null;}
  abstract public void draw();
}
```

```
class Line extends Shape {
  public void draw() {
    System.out.print("Line");
  }
}
```

```
class Rect extends Shape {
  public void draw() {
    System.out.print("Rect");
  }
}
```

```
class Circle extends Shape {
  public void draw() {
    System.out.print("Circle");
  }
}
```

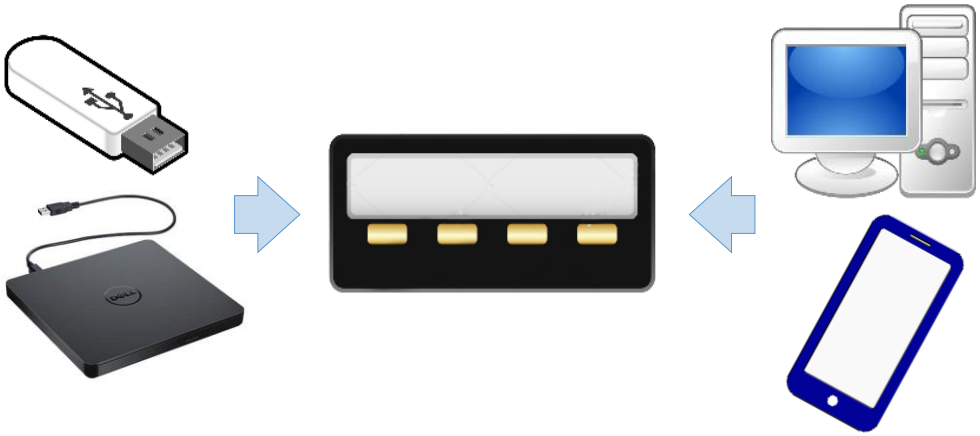
# 실습

```
abstract class Shape {

 int x, y;
 public void translate(int x, int y) {
   this.x = x;
   this.y = y;
 }
 public abstract void draw();
};

class Rectangle extends Shape {
 int width, height;
 public void draw() {  System.out.println("사각형 그리기 메소드");
 }
};
```

```
class Circle extends Shape {
 int radius;
 public void draw() {  System.out.println("원 그리기 메소드");
 }
};

public class AbstractTest {
 public static void main(String args[]) {
  // Shape s1 = new Shape(); // 오류!!
  Shape s2 = new Circle(); // OK!!
  s2.draw();
 }
};
```

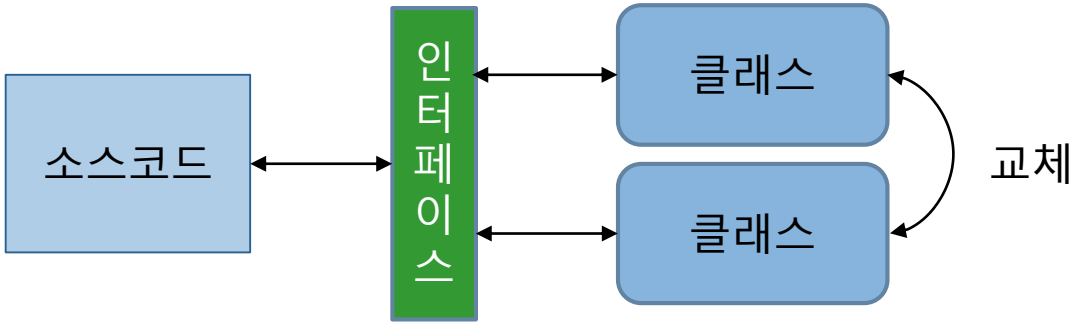
# 인터페이스interface (1)

- 시그너처signature가 변하지 않는 것을 전제로 하여 다형성을 정의하는 객체간의 규약
  - 클래스의 상세 설계를 몰라도 사용 가능
  - 인터페이스의 메소드만 알고 있으면 가능



# 인터페이스interface (2)

- 역할
  - 기능 명세서
  - 작성 코드가 클래스에 종속되지 않도록 함
    ⇒ 클래스를 쉽게 교체 가능



# 인터페이스 구성 가능 멤버

Java 8 기준

- **상수 필드**
  - 모든 필드는 public static final로 가정
- **추상 메소드**
  - 추상 메소드는 public abstract로 가정
- 디폴트 메소드
  - 실행 블록을 가지는 메소드
  - default 키워드 반드시 사용
  - public으로 가정
- 정적 메소드
  - public으로 가정

```java
public interface Clock {
  (abstract public) int getMinute();
  (abstract public) int getHour();
  (abstract public) void setMinute(int i);
  (abstract public) void setHour(int i);
}
```

```java
public interface RemoteController {
  (public static final) int MIN_VOLUMN = 0;
  (public static final) int MAX_VOLUMN = 100;

  (abstract public) void UpVolumn();
  (abstract public) void DownVolumn();

  (public) default void GetVolumn() {
   System.out.println("현재 볼륨을 확인합니다.");
  }
}
```

# 인터페이스 구현

- implements 키워드 사용

```
public interface Clock {
  (abstract public) int getMinute();
  (abstract public) int getHour();
  (abstract public) void setMinute(int i);
  (abstract public) void setHour(int i);
}

public class DigitalClock implements Clock {
  public int getMinute() {· · ·};
  public int getHour() {· · ·};
  public void setMinute(int i) {· · ·};
  public void setHour(int i) {· · ·};

  // 메소드 추가
  public int getSecond() {· · ·};
  public void setSecond(int i) {· · ·};
}
```

# 실습

```
interface RemoteControl {
 void turnOn();
 void turnOff();
 public default void printBrand()
{ System.out.println("Remote Control 가능 TV"); }
}

class Television implements RemoteControl {
 boolean on;
 public void turnOn() {
   on = true;
   System.out.println("TV가 켜졌습니다.");
 }
 public void turnOff() {
   on = false;
   System.out.println("TV가 꺼졌습니다.");
 }
 @Override
 public void printBrand() {
   System.out.println("Power Java TV입니다."); }
}
```

```
public class TestInterface {
  public static void main(String args[]){
    RemoteControl obj = new Television();
    obj.turnOn();
    obj.turnOff();
    obj.printBrand();
  }
}
```

# 실습

- 영화예매 예제의 DiscountPolicy를 Interface로 변경하시오.

# 실습

```
public interface DiscountPolicy {
     int discount(int 원가격);
}
```

```
public class FixDiscountPolicy implements DiscountPolicy{
    private int 할인금액;

    public FixDiscountPolicy(int 할인금액) {
        this.할인금액 = 할인금액;
    }
    @Override
    public int discount(int 원가격) {
        return 원가격 - 할인금액;
    }
}
```

```
public class RateDiscountPolicy implements DiscountPolicy{
    double 할인률;

    public RateDiscountPolicy(double 할인률) {
        this.할인률 = 할인률;
    }
    @Override
    public int discount(int 원가격) {
        return (int)(원가격 * (1 - 할인률));
    }
}
```

# 인터페이스의 특징

- 객체 생성 불가
  - 레퍼런스 변수 타입으로 사용 가능
- 다중 상속 가능
- 다중 구현 가능
- 동시에 상속과 구현 가능

# 다중 상속 지원

```
public interface Clock {
   . . .
}
```

```
public interface RemoteController {
   . . .
}
```

```
public interface ClockRemoteController extends
                        Clock, RemoteController {
   . . .
}
```

# 다중 구현 지원

```
public interface Clock {
   . . .
}
```

```
public interface RemoteController {
   . . .
}
```

```
public class ClockRemoteController implements
                        Clock, RemoteController {
   . . .
}
```

# 상속 및 구현 동시 적용

```
public class Phone {
 public void SendCall() {· · ·}
 public void ReceiveCall() {· · ·}
}
```

```
public interface MP3 {
  (abstract public) void play();
  (abstract public) void stop();
}
```

```
public class SmartPhone extends Phone implements MP3 {
   . . .
}
```

# 추상 클래스와 인터페이스 비교

| 비교 | 내용 |
| --- | --- |
| 추상 클래스 | • 상수, 변수 필드 포함 가능<br>• 모든 서브 클래스에 공통된 메소드가 있는 경우는 추상 클래스가 적합<br>• 단일 상속만 지원 |
| 인터페이스 | • 상수 필드만 포함 가능<br>• 객체 간의 규약을 정의하는 경우에는 인터페이스 사용<br>• 다중 상속 지원 |

# 추상클래스 vs 인터페이스

- 인터페이스와 추상클래스의 두드러진 차이
  - 추상클래스는 구현된 메소드를 포함 할수 있고 인터페이스는 그렇지 못하다는 것입니다. 하지만 JAVA 8부터는 default interface method 기능을 제공해 구현된 메서드를 포함할 수 있습니다.
  - 추상클래스로 정의된 타입을 구현하는 클래스는 반드시 추상클래스의 하위 클래스가 되야 한다는 것입니다. 이와 달리 인터페이스를 구현하는 클래스의 경우는, 인터페이스에 정의된 모든 메소드를 구현하고 인터페이스 구현 계약을 지키면 됩니다.
    - 이건 클래스 상속과는 무관합니다. 자바에서 클래스 간의 상속은 단일 상속만을 허용하기때문에 추상 클래스로 타입을 정의할때는 심한 제약이 따릅니다.

→기본적으로 interface를 사용하되, 자식이 공통된 필드를 가지고 있다면 추상클래스를 사용하는 것을 고려하는 것이 일반적!

# 인터페이스는 왜 쓰는가?

- 개발시간의 단축
  - 실제 인터페이스를 구현하는 사람과 가져다 쓰는 사람이 동시에 개발 진행 가능 (http://xxxelppa.tistory.com/77)
- 표준화 가능
  - 프로젝트에 사용되는 기본 틀을 인터페이스로 작성한 후, 개발자에게 인터페이스를 구현하여 프로그램을 작성토록 하여, 보다 일관되고 정형화된 프로그램 개발 가능
- 서로 관계없는 클래스를 연결 시켜줌 (strategy pattern)
  - 아무 관계도 없지만 하나의 인터페이스를 공통으로 구현함으로써 관계를 맺어줄 수 있음 (http://debasishg.blogspot.kr/2008/06/scala-to-java-smaller-inheritance.html)
- 독립적인 프로그래밍이 가능
  - 선언과 구현을 분리 → 독립적인 프로그램 작성 가능
  - 결과적으로 한 클래스의 변경이 다른 클래스에 영향을 미치지 않음
  - E.g., Map interface (HashMap or TreeMap or SuperHashMap)

# 인터페이스는 언제 제일 좋은가?

- **인터페이스는 믹스인(mixin)을 정의하는데 이상적이다.**
  - 믹스인은 클래스가 자신의 "본래타입"에 추가하여 구현할 수 있는 타입으로써, 선택 가능한 기능을 제공하며, 그 기능을 제공 받고자 하는 클래스에서 선언함.
  - 예를 들어, Comparable은 믹스인 인터페이스로써, 상호 비교 가능한 다른 객체와의 비교를 통해 클래스의 인스턴스가 정렬된다는 것을 그 클래스에서 선언 할 수 있다.
  - 그런 인터페이스를 믹스 인이라고 부르는 이유는, 어떤 타입의 본래 기능에 선택 가능한 기능을 "섞는(mixed in)"것이 가능하기 때문이다.
  - 기존 클래스가 추상 클래스로부터 상속받기 위해 개조될 수 없는 것과 같은 이유로, 추상클래스는 믹스 인을 정의하는데 사용될 수 없다.
    - 즉, 클래스는 하나 이상의 수퍼 클래스를 가질 수 없으므로, 클래스 상속 계층에 믹스 인을 끼워 넣을 만한 마땅한 곳이 없기 때문이다.

# 다음 강의 주제

- Class Diagram in UML